

Dossier Técnico — RenamerJF ML (KP · Medical Records Manager)

Propósito de este documento: material técnico fuente, completo y autocontenido, para alimentarlo a un asistente (Claude) y generar un documento formal de presentación (estilo ISO/IEC — p. ej. ISO/IEC/IEEE 12207 ciclo de vida del software, o ISO/IEC 25010 calidad de producto). Reúne arquitectura, stack, módulos, modelo de datos, API, seguridad, privacidad y distribución. Última actualización: 2026-06-22.

1. Resumen ejecutivo

RenamerJF ML (marca de interfaz: **KP**) es una aplicación de escritorio para bufetes de abogados especializados en lesiones personales (*personal injury*). Automatiza la gestión y el renombrado de documentos médicos (facturas, expedientes, reportes) mediante extracción inteligente de datos: identifica el proveedor médico, extrae fechas de servicio y extrae campos de facturación (cargos, ajustes, pagos PIP/seguro, saldo pendiente).

El sistema combina tres capas de inteligencia en cascada, de menor a mayor costo:

1. **Reglas / expresiones regulares + coincidencia difusa** (local, instantáneo, gratis).
2. **Modelo de Machine Learning propio** (DistilBERT NER, ejecutado localmente vía ONNX).
3. **IA generativa externa** (Google Gemini), solo como último recurso y con consentimiento explícito del usuario.

Es una aplicación **local-first**: el backend y la base de datos corren en la propia máquina del usuario dentro del contenedor de Electron; no requiere servidor central. Se distribuye como instalador para **Windows (.exe)** y **macOS (.dmg)**.

2. Contexto y problema de negocio

Los bufetes de lesiones personales manejan grandes volúmenes de documentos médicos por cada caso.

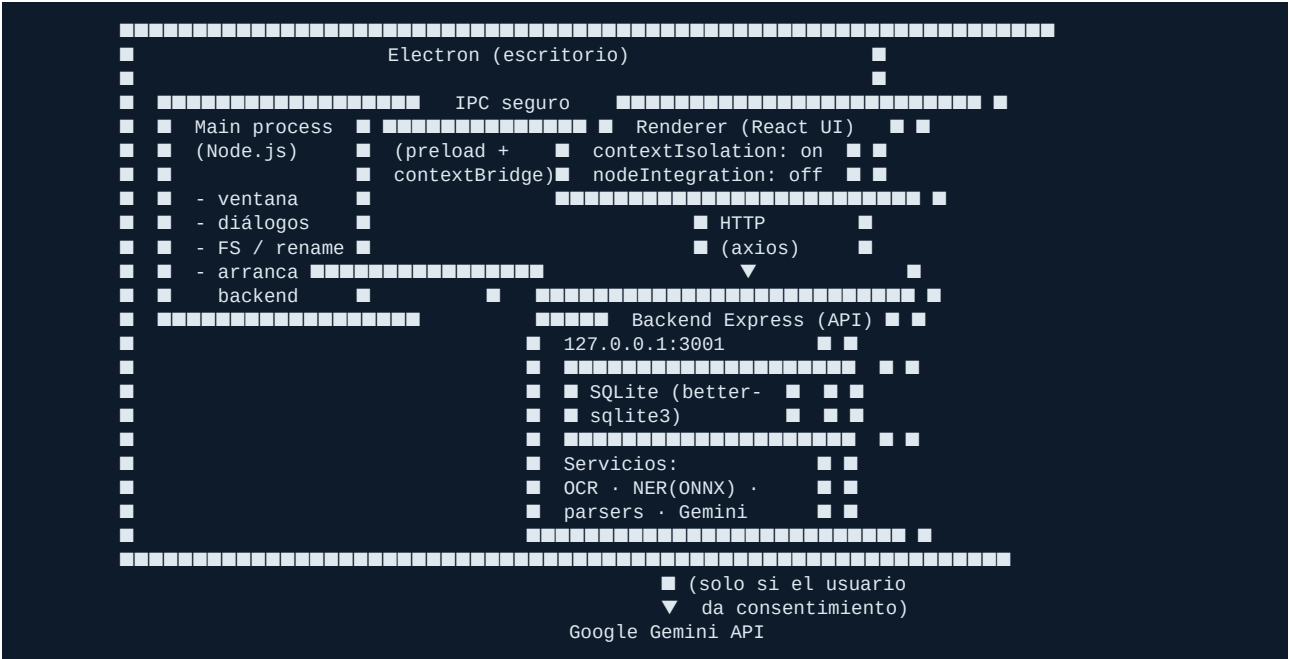
Cada documento debe:

- Renombrarse con una convención consistente (código de tipo + proveedor + fechas).
- Asociarse a un proveedor médico conocido.
- En el caso de facturas, totalizarse (cargos, ajustes, pagos, saldo) para el cálculo de daños.

Hacerlo manualmente es lento y propenso a errores. RenamerJF ML automatiza la lectura del documento (incluyendo PDFs escaneados vía OCR), sugiere el proveedor y los datos, y genera el nuevo nombre y los totales de facturación, manteniendo además un registro histórico y un seguimiento de casos.

3. Arquitectura general

Aplicación de escritorio multiproceso basada en **Electron**, con tres componentes lógicos que conviven en el mismo equipo:



Puntos clave de la arquitectura:

- El **main process** de Electron arranca el backend Express **en proceso** (en producción) y lo expone únicamente en **127.0.0.1** (loopback), nunca en la red.
- El **renderer** (React) está aislado: **contextIsolation: true**, **nodeIntegration: false**, y se comunica con el sistema de archivos solo a través de un **preload** con **contextBridge** (IPC).
- La **base de datos** SQLite se almacena en el directorio **userData** del sistema operativo, de modo que sobrevive a las actualizaciones de la aplicación.
- El **JWT_SECRET** se genera aleatoriamente (32 bytes) en el primer arranque y persiste, de modo que los tokens de sesión siguen siendo válidos tras reinicios/actualizaciones.

4. Stack tecnológico

Capa	Tecnología	Versión
Shell de escritorio	Electron	33.x
Empaquetado / instaladores	electron-builder	26.x (NSIS para Win, DMG para Mac)
Frontend	React + Vite	React 19 · Vite 8
Cliente HTTP	axios	1.x
Routing UI	react-router-dom	7.x
Import/Export datos	papaparse (CSV) · xlsx (Excel)	5.x · 0.18
Backend	Node.js + Express	Express 5.x

Capa	Tecnología	Versión
Base de datos	SQLite vía better-sqlite3	12.x
Autenticación	jsonwebtoken (JWT) · bcryptjs	9.x · 3.x
Extracción texto PDF	pdf-parse · pdfjs-dist	1.x · 5.x
OCR	tesseract.js	7.x
Render de imágenes (OCR)	canvas	3.x
Coincidencia de proveedores	fuse.js (búsqueda difusa)	7.x
Inferencia ML (producción)	onnxruntime-node (ONNX Runtime)	—
Entrenamiento ML	Python · HuggingFace Transformers · PyTorch · DistilBERT	transformers 4.57 · torch 2.8
Datos sintéticos	Synthea	—
IA generativa (fallback)	Google Gemini API (@ google/generative-ai)	0.24

Nota: Python y PyTorch **solo** se requieren para *re-entrenar* el modelo. En producción la inferencia se ejecuta con [onnxruntime-node](#), sin dependencia de Python.

5. Módulos funcionales

Módulo (UI)	Descripción
Single File (Renamer)	Carga un documento, lo analiza, sugiere proveedor/tipo/fechas y genera el nuevo nombre.
Batch Renamer	Procesa múltiples archivos a la vez con la misma convención de nombres.
Cases (Case Tracker)	Gestión de metadatos de casos (número, partes, fecha del siniestro, fases, fechas clave, notas). Importación masiva.
Billing	Extrae y totaliza campos de facturación de un documento médico; calculadora de daños.
Medical Contacts (Providers)	Catálogo de proveedores médicos (datos de contacto, portal, notas). Importación CSV.
AI Chat	Permite hacer preguntas en lenguaje natural sobre un documento cargado (vía Gemini, con consentimiento).
Account / Admin	Gestión de cuenta propia y administración de usuarios (solo admin).

Componentes de frontend relevantes: [FileRenamer](#), [BatchRenamer](#), [BillingPanel](#), [BillingCalculator](#), [CaseTracker](#), [ProviderList/ProviderForm/ProviderCard](#), [ChatPanel](#), [FilePreview](#), [AIConsentModal](#), [AccountPanel](#), [Login](#), [Dashboard](#).

6. Pipeline de identificación de proveedor (módulo Renamer)

POST /api/analyze ejecuta:

1. **Extracción de texto** del archivo: capa de texto del PDF ([pdf-parse](#)); si el PDF tiene menos de ~50 caracteres se asume escaneado y se recurre a **OCR** (Tesseract); las imágenes van directo a OCR.
2. **Coincidencia de proveedor** contra el catálogo local mediante reglas + búsqueda difusa ([fuse.js](#)), con conteo de ocurrencias y límites de palabra para nombres cortos.
3. **Extracción de fechas** (fecha de servicio inicio/fin, fecha de actualización) y banderas (p. ej. PIP agotado).
4. **Escalado a IA (Gemini)** únicamente si la confianza local < 0.35 y el usuario ha dado consentimiento ([allowAI=true](#)). Si la confianza es baja pero no hay consentimiento, la API responde [needsAI=true](#) para que la UI muestre el modal de consentimiento.
5. **Sesión de chat**: el texto extraído se guarda en una sesión del lado del servidor para que el módulo de chat pueda responder preguntas sin volver a subir el archivo. El texto extraído se elimina de la respuesta HTTP (vive solo en la sesión server-side).

7. Pipeline de facturación — cascada de confianza (módulo Billing)

POST /api/billing/analyze aplica una **cascada de extractores** ordenados de mayor a menor confianza; se detiene en el primero que supere el umbral:

```
Texto del PDF
  ■■ Regex Athena Health      (alta confianza)
  ■■ Regex de tabla resumen
  ■■ Partidas CPT (line items)
  ■■ Regex de totales hospitalarios
  ■■ ML-NER (DistilBERT ONNX)  ← se activa si la confianza < ~70 %
  ■■ Gemini AI                  ← se activa si la confianza < 35 % + consentimiento del usuario
```

Campos extraídos y almacenados: [total_charges](#), [total_adjustments](#), [pip_paid](#), [health_ins_paid](#), [patient_paid](#), [outstanding](#), además de [confidence](#) y [source](#) (local / ml / ai). Los resultados pueden guardarse (POST /api/billing/save) y consultarse por número de caso.

8. Modelo de Machine Learning — DistilBERT NER

- **Tarea:** Reconocimiento de Entidades Nombradas (NER) sobre texto de facturación médica (clasificación de tokens).
- **Modelo base:** [distilbert-base-uncased](#), *fine-tuned*.
- **Etiquetas (7):** 0, B-CHARGE, B-ADJUST, B-PIP, B-HEALTH, B-PATIENT, B-OUTSTANDING.
- **Longitud máxima de secuencia:** 256 tokens.
- **Métrica reportada:** F1 = 0.965.
- **Datos de entrenamiento:** 100 % sintéticos, generados con **Synthea** (generador de

registros de pacientes sintéticos). *Nunca* se usan datos reales de pacientes.

- **Inferencia en producción:** el modelo se exporta a **ONNX** y se ejecuta con `onnxruntime-node`. Se incluye un **tokenizador WordPiece propio en JavaScript** que lee `vocab.txt` directamente, eliminando cualquier dependencia de Python/HuggingFace en runtime.
 - **Pipeline de entrenamiento (Python):** `prepare_dataset.py` → `train.py` → `export_onnx.py`, con dependencias fijadas (transformers 4.57.6, torch 2.8.0, onnx 1.19.1, etc.).
 - Los pesos del modelo (~253 MB) se excluyen del repositorio (`.gitignore`) por tamaño y se re-generan localmente; en el build se empaquetan como recursos extra.
-

9. OCR

- **Motor:** Tesseract.js (con `eng.traineddata`).
 - **Disparadores:** PDF con capa de texto insuficiente (< ~50 caracteres) o archivos de imagen (`.jpg`, `.jpeg`, `.png`, `.tiff`, `.tif`, `.bmp`, `.webp`).
 - Soporta OCR de múltiples páginas para documentos de facturación extensos.
-

10. IA generativa (Google Gemini) — uso y consentimiento

- Es el **último recurso** de las cascadas de análisis y facturación, y el motor del chat.
 - **Solo se invoca** cuando: (a) la confianza local es baja, (b) existe una clave `GEMINI_API_KEY` configurada, y (c) el usuario ha **consentido explícitamente** para esa sesión (`allowAI=true`, gestionado por `AIConsentModal`).
 - **Implicación de privacidad:** cuando se invoca, el texto extraído del documento sale del equipo hacia un tercero (Google). Esto debe gobernarse por la política de privacidad/cumplimiento del bufete (ver §13).
 - **Chat de documento:** el módulo de chat usa la **API de Gemini**. Antes de enviar cualquier mensaje, el diálogo muestra una **advertencia de privacidad/PII** indicando que las preguntas y el texto del documento se envían a Google, y que se evite incluir información personal/de salud innecesaria. La consulta a Gemini es **opt-in** (botón "Enable Gemini").
 - **Migración planificada (Lois de Filevine):** el proveedor de IA previsto a futuro para el chat es **Lois**, el asistente de **Filevine**. Al mantener los datos dentro de la plataforma de gestión de casos del bufete, evita enviar PHI a Google. La integración conservará la misma interfaz interna (`chatWithDocument(sessionId, messages)` → `reply`), reemplazando solo la llamada a Gemini (ver `backend/src/services/aiChat.js`).
-

11. Modelo de datos (SQLite)

Tabla	Propósito	Campos destacados
users	Cuentas de usuario	id (UUID), email (único), password_hash (bcrypt), role (admin/user)
providers	Catálogo de proveedores médicos	name, type, specialty, phone, fax, email, address, portal_url, notes
document_types	Tipos de documento (prefijos de nombre)	code (B,MR,PD,LT,RX,IN,OT), label
rename_history	Historial de renombrados	original_name, new_name, dos_start/end, update_date, pip_exhausted, FKs a usuario/proveedor/tipo
billing_summaries	Resúmenes de facturación	totales (cargos, ajustes, PIP, seguro, paciente, saldo), confidence, source
cases	Seguimiento de casos	num (único), partes (first/last), dol (fecha del siniestro), fases, fechas clave, notes

Datos semilla en primer arranque: tipos de documento, y usuarios iniciales desde variables de entorno (`SEED_*`). Si no hay usuarios, la app ofrece *bootstrap* del primer administrador.

12. API REST (backend Express)

Base: `http://127.0.0.1:3001/api`. Todos los endpoints requieren JWT salvo los marcados como *público*.

Auth (`/auth`)

- `GET /status` (*público*) — indica si la instalación necesita bootstrap.
- `POST /bootstrap` (*público, solo si no hay usuarios*) — crea el primer admin.
- `POST /login` (*público*) — devuelve JWT (válido 8 h).
- `POST /change-password` — el usuario cambia su propia contraseña.
- `POST /register` (*admin*) — alta de usuario.
- `GET /users` (*admin*) — lista de cuentas (sin hashes).
- `POST /reset-password` (*admin*) — restablece la contraseña de otro usuario.
- `DELETE /users/:id` (*admin*) — elimina cuenta (no a sí mismo; no al último admin).

Análisis y facturación

- `POST /analyze` — análisis de documento (proveedor, fechas, sesión de chat).
- `POST /billing/analyze` — extracción de facturación (cascada).
- `POST /billing/save` — guarda resumen de facturación.
- `GET /billing/:caseNum` — resúmenes por caso.
- `POST /chat` — pregunta en lenguaje natural sobre la sesión de un documento.

Proveedores (`/providers`)

- `GET /`, `GET /:id`, `POST /suggest` (autenticado); `POST /`, `PUT /:id`, `DELETE /:id`, `POST /import` (*admin*).

Casos (`/cases`)

- `GET /`, `POST /`, `DELETE /:num`, `POST /import` (todos autenticados).

Historial (`/history`)

- `GET /`, `POST /` (autenticados).

Referencia

- `GET /document-types` (público) — datos de referencia estáticos.
-

13. Seguridad y privacidad

Controles implementados:

- **Contraseñas:** hash con `bcrypt` (salt, factor 10); nunca en texto plano.
- **Sesiones:** `JWT` firmado con secreto aleatorio persistente, expiración de 8 h.
- **Roles:** middleware `admin` separa operaciones de gestión de usuarios y catálogos.
- **Inyección SQL:** todas las consultas usan sentencias parametrizadas (`better-sqlite3`).
- **Sin secretos en el repositorio:** `.env` está en `.gitignore`; se provee `.env.example`.
- **Endurecimiento de Electron:** `contextIsolation: true`, `nodeIntegration: false`, carga de contenido local (sin contenido remoto), comunicación FS solo vía preload/IPC.
- **Login sin enumeración de usuarios** (mismo error para usuario inexistente y contraseña incorrecta).
- **Protecciones de cuentas:** auto-registro deshabilitado (solo admins crean usuarios), no se puede eliminar la propia cuenta ni al último administrador, bootstrap del primer admin bloqueado tras el setup.

Endurecimientos recientes (junio 2026):

- El módulo de **Casos** ahora exige token en todos sus endpoints (contienen PHI: nombres, fecha del siniestro, notas).
- El backend escucha **solo en 127.0.0.1** (no accesible desde la red local).
- Los endpoints de análisis y facturación validan la **extensión del archivo** (lista blanca de tipos de documento/imagen), evitando la lectura de archivos arbitrarios del host.
- **Rate limiting** en `/login` (10 intentos / 15 min por IP) para mitigar fuerza bruta.
- **Cabeceras de seguridad** vía `helmet` (nosniff, frameguard, HSTS, etc.).

Consideraciones de cumplimiento (a evaluar por el bufete):

- **PHI a terceros:** el uso de Gemini envía texto de documentos a Google. Requiere decisión de negocio sobre privacidad/cumplimiento (p. ej. HIPAA en EE. UU. y, en su caso, un *Business Associate Agreement* con el proveedor de IA).
- **Datos de entrenamiento:** el modelo ML se entrena exclusivamente con datos **sintéticos** (Synthea); no se usan registros reales de pacientes para entrenar ni se envían a LLMs en el entrenamiento.
- **Transporte local:** la comunicación interna es HTTP en loopback (no expuesta a la red).

Mejoras de seguridad recomendadas (backlog):

- Auditoría de accesos (quién consulta/edita qué y cuándo).
- Expiración de sesión configurable y revocación de tokens.

14. Distribución, build y CI/CD

- **Instaladores:** Windows **NSIS (.exe)** x64 y macOS **DMG** (arm64 + x64).
- **Iconografía:** **icon.ico** multi-resolución (16–256 px) para Windows e **icon.icns** para macOS; iconos de instalador/desinstalador declarados explícitamente en la configuración NSIS.
- **CI:** workflow de GitHub Actions (**build-windows.yml**) con *jobs* paralelos en **macos-latest** y **windows-latest**; ejecución manual (**workflow_dispatch**). Ejecuta **release:mac** / **release:win**, que publican instaladores y metadatos de actualización a un *release* de GitHub (además de subir los artefactos).
- **Auto-actualización:** mediante **electron-updater** contra GitHub Releases. Las apps instaladas verifican una versión más nueva al iniciar, la descargan en segundo plano y piden reiniciar para aplicarla (solo en producción; los errores se ignoran para no interrumpir el uso).
Proceso de release: (1) subir la versión en **package.json**; (2) correr el workflow → crea un *release* en borrador; (3) publicar el borrador para que los clientes lo reciban.
- **Persistencia de datos:** la base de datos se ubica en el **userData** del SO para sobrevivir a actualizaciones.
- **Nombre de app distintivo** (**RenamerJF ML**) para aislar el **userData** de otras variantes.

15. Estructura del proyecto

```

  ├── electron/      Shell de escritorio (main process + preload)
  ├── frontend/     App React (Vite)
  │   ├── src/
  │   │   ├── components/  FileRenamer, BatchRenamer, BillingPanel, CaseTracker, ...
  │   │   ├── pages/      Dashboard, Login
  │   │   ├── services/    api.js (cliente axios con JWT automático)
  │   ├── backend/        API Express
  │   │   ├── src/
  │   │   │   ├── routes/  analyze, billing, auth, cases, chat, providers, history
  │   │   │   ├── services/ billingParser, docAnalyzer, billingAI, aiAnalyzer, aiChat, ocr
  │   │   │   ├── middleware/ auth.js (JWT + admin)
  │   │   │   └── db/       schema.js (SQLite)
  │   └── ml/             Pipeline de ML
  │       ├── scripts/    prepare_dataset.py, train.py, export_onnx.py
  │       ├── models/     infer.js, ner_config.json, tokenizer/, bill-ner.onnx
  │       └── synthea/     generador de datos sintéticos
```

16. Atributos de calidad (mapeo orientativo a ISO/IEC 25010)

Característica	Cómo lo aborda el sistema
Adecuación funcional	Cascadas de extracción con respaldo ML e IA; cobertura de PDF, escaneados e imágenes.

Característica	Cómo lo aborda el sistema
Eficiencia de desempeño	Resolución local primero; IA externa solo como último recurso.
Compatibilidad	Instaladores nativos para Windows y macOS.
Usabilidad	UI de escritorio unificada; vista responsive consistente entre SO.
Fiabilidad	Base de datos local persistente en <code>userData</code> ; transacciones en importaciones.
Seguridad	Auth JWT + bcrypt, roles, SQL parametrizado, backend en loopback, lista blanca de archivos.
Mantenibilidad	Separación clara front/back/ML; pipeline de re-entrenamiento documentado.
Portabilidad	Inferencia ML sin Python (ONNX); empaquetado multiplataforma.

17. Limitaciones y trabajo futuro

- El análisis depende de la calidad del OCR en documentos escaneados de baja resolución.
- El uso de IA externa está condicionado a configuración de clave y consentimiento; sin ella, el sistema opera solo con capas local + ML.
- Backlog de seguridad: *rate limiting*, `helmet`, y validación de ruta base de archivos.
- Posible evolución: sincronización opcional entre equipos, auditoría de accesos, y ampliación del conjunto de etiquetas del modelo NER.